Insertion in Circular Doubly Linked List (CDLL)

Types of Insertion

1. **Insertion at Beginning**
2. **Insertion at End**
3. **Insertion at Specific Position**

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

/* Insert at Beginning */
struct Node* insertBegin(struct Node *head)
{
    struct Node *newnode, *last;
    int value;

    newnode = (struct Node*)malloc(sizeof(struct Node));

    printf("Enter data: ");
    scanf("%d", &value);

    newnode->data = value;

    // If list is empty
    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        head = newnode;
        return head;
    }

    last = head->prev;

    newnode->next = head;
    newnode->prev = last;

    last->next = newnode;
```

```
    head->prev = newnode;

    head = newnode;

    return head;
}

/* Insert at End */
struct Node* insertEnd(struct Node *head)
{
    struct Node *newnode, *last;
    int value;

    newnode = (struct Node*)malloc(sizeof(struct Node));

    printf("Enter data: ");
    scanf("%d", &value);

    newnode->data = value;

    // If list is empty
    if(head == NULL)
    {
        newnode->next = newnode;
        newnode->prev = newnode;
        head = newnode;
        return head;
    }

    last = head->prev;

    newnode->next = head;
    newnode->prev = last;

    last->next = newnode;
    head->prev = newnode;

    return head;
}

/* Insert at Specific Position */
struct Node* insertPosition(struct Node *head)
{
    struct Node *newnode, *temp;
    int value, pos, i;

    printf("Enter position: ");
    scanf("%d", &pos);
```

```
    printf("Enter data: ");
    scanf("%d", &value);

    newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->data = value;

    // Insert at beginning
    if(pos == 1)
    {
        return insertBegin(head);
    }

    temp = head;

    for(i = 1; i < pos-1; i++)
    {
        temp = temp->next;

        // If reached head again → invalid position
        if(temp == head)
        {
            printf("Invalid Position\n");
            return head;
        }
    }

    newnode->next = temp->next;
    newnode->prev = temp;

    temp->next->prev = newnode;
    temp->next = newnode;

    return head;
}

/* Display CDLL */
void display(struct Node *head)
{
    struct Node *temp;

    if(head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    temp = head;

    printf("CDLL: ");
```

```
    do
    {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    while(temp != head);

    printf("(back to head)\n");
}

/* Main Function */
int main()
{
    struct Node *head = NULL;
    int choice;

    while(1)
    {
        printf("\n----- MENU -----\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                head = insertBegin(head);
                break;

            case 2:
                head = insertEnd(head);
                break;

            case 3:
                head = insertPosition(head);
                break;

            case 4:
                display(head);
                break;

            case 5:
                exit(0);
```

```
            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

```
----- MENU -----

1. Insert at Beginning

2. Insert at End

3. Insert at Position

4. Display

5. Exit

Enter choice: 1

Enter data: 10

Enter choice: 2

Enter data: 20

Enter choice: 4

CDLL: 10 <-> 20 <-> (back to head)
```